

Admin DSL React Widget IR Catalog

Executive Summary

This document is the first concrete artifact for rebuilding the current Admin DSL renderer from the bottom up. The current renderer in `web/src/admin-dsl/render.tsx` is one large interpreter that renders most Admin DSL constructs with inline `div`, `table`, `button`, and CSS style objects. That was useful for rapid iteration, but it is not the shape we want for a generated or compiler-assisted UI DSL system. We want a clear catalog of React widgets first, before writing or refactoring React implementation code.

The artifact in this document is a **React Widget IR**. It lists the widgets that should exist, their atomic design classification, their props, their action slots, their file layout, and the Storybook stories needed to test them. It is an intermediate representation: downstream passes can consume it to generate file scaffolds, Storybook skeletons, renderer adapter maps, prop contracts, tests, and implementation prompts. The artifact does not have to be final React code. It needs to be deterministic, reviewable, and detailed enough for a human or LLM pass to elaborate.

The main architectural correction is that passes are not necessarily consecutive steps in one fixed pipeline. A pass is defined by the artifacts it requires and the artifacts it produces. For example, a widget extraction pass can require `render.tsx`, `schema.ts`, `builder.ts`, and Storybook fixtures, then produce this widget catalog. A Storybook planning pass can require this catalog and produce story files. A file-layout pass can require this catalog and produce scaffold paths. An LLM elaboration pass can require this catalog and produce implementation TODOs, example scenarios, and Playwright sketches.

Problem Statement

The current Admin DSL v2 renderer has too many responsibilities in one file. It interprets node kinds, extracts untyped props, chooses layout, applies visual styling, normalizes action presentation, dispatches events, renders mobile breakpoints, and implements data widgets such as tables, comparison tables, calendars, image galleries, forms, and shells.

This creates several problems:

- It is difficult to reason about one widget in isolation because the implementation is embedded inside a large switch statement.
- Storybook coverage is scenario-oriented, but the widget-level test matrix is implicit.
- Props are discovered by reading render branches instead of being declared as widget contracts.
- Action behavior is expressed through generic `AdminActionRef` lists and placement strings instead of typed action slot callbacks.
- File layout does not reflect component boundaries.
- Future generated renderer scaffolding has no obvious target because there are no explicit widget modules to generate around.

The immediate goal is not to implement the new React widgets. The immediate goal is to define the target widget inventory. This document is the IR that later implementation passes should consume.

Proposed Solution

Extract the current Admin DSL constructs into an explicit component system:

Current artifacts

- web/src/admin-dsl/render.tsx
- web/src/admin-dsl/calendar.tsx
- web/src/admin-dsl/schema.ts
- web/src/admin-dsl/builder.ts
- web/src/admin-dsl/actions.ts
- web/src/admin-dsl/AdminDslWorkbench.stories.tsx

|
v

React Widget IR Catalog

- widget names
- atom/molecule/organism classification
- props
- action slots / callback signatures
- supported Admin DSL constructs
- file path layout
- Storybook scenario matrix
 - +--> file scaffold pass
 - +--> Storybook skeleton pass
 - +--> renderer adapter pass
 - +--> prop contract generation pass
 - +--> implementation prompt pass
 - +--> Playwright scenario sketch pass

The widgets should be organized by atomic design level:

- **Atom:** A primitive control or visual unit that does not own domain layout. Examples: ActionButton, StatusText, FieldLabel, TextInput, SwitchControl.
- **Molecule:** A small composition of atoms with a local behavior. Examples: ActionGroup, SearchBox, Tabs, MetricCard, KeyValueList, FormErrorSummary.
- **Organism:** A larger semantic UI region that maps to an Admin DSL construct or a major renderer target. Examples: WorkbenchShell, PageHeader, Panel, ResourceTable, ComparisonTable, MonthCalendar, Form, OverlaySurface.

Shells are classified as organisms with role: shell. They are larger than a normal organism, but using a single three-level classification keeps the IR simple.

Artifact-Pass Model

The following pass definitions show how this document fits into the broader meta-DSL direction. These passes are not required to run in this order. Each pass is defined by required artifacts and produced artifacts.

Pass	Requires	Produces	Notes
Current renderer inventory pass	render.tsx, calendar.tsx, schema.ts, builder.ts	AdminDslConstructInventory	Extracts current node kinds, props, actions, and render branches.
Widget extraction pass	AdminDslConstructInventory, Storybook fixtures	ReactWidgetIRCatalog	This document is the first hand-authored version of that artifact.

Pass	Requires	Produces	Notes
File layout pass	ReactWidgetIRCatalogWidgetFileScaffoldPlan		Limits directories and file names, not implementation.
Storybook plan pass	ReactWidgetIRCatalogStorybookScenarioPlan current stories		Limits one story plan per widget plus integration scenarios.
Renderer adapter pass	ReactWidgetIRCatalogRendererRegistryPlan Admin DSL schema		Maps Admin DSL constructs to widgets and prop adapters.
Action typing pass	ReactWidgetIRCatalogActionSlotSignatureIR action schema		Converts generic action placements into typed slot callback signatures.
LLM elaboration pass	Any of the above artifacts	implementation prompts, Playwright sketches, edge-case lists	Output may be semi-formal and reviewed by humans before code generation.

A deterministic pass does not need to produce finished code. It needs to produce a stable artifact that can be reviewed and consumed by later deterministic or LLM-assisted passes.

Shared Widget Types

Every widget should avoid directly depending on raw `AdminNode` when possible. Renderer adapters can translate from Admin DSL nodes into widget props. Widgets receive typed props and callbacks.

Base action types

The current system uses `AdminActionRef` from `web/src/admin-dsl/schema.ts`. The widget IR should keep that runtime action shape as an implementation detail but expose contextual callbacks.

```
export interface ActionViewModel {
  id?: string;
  type: "open" | "close" | "navigate" | "mutation" | "confirm" | "refresh" | "upload";
  target: string;
  label: string;
  intent?: "neutral" | "primary" | "danger";
  priority?: "primary" | "secondary" | "tertiary";
  presentation?: "button" | "icon" | "menuItem" | "overflow" | "link";
  disabled?: boolean;
  loading?: boolean;
  accessibilityLabel?: string;
}
```

Action slot callback signatures

Actions should be defined by their context signature. A row action is an action that receives table and row context. A bulk action receives table and selected-row context. A form submit

action receives form values. The renderer can still lower these callbacks to dispatchAdminAction internally.

```
export type PageActionHandler = (action: ActionViewModel, context: {
  pageId?: string;
}) => void;

export type PanelActionHandler = (action: ActionViewModel, context: {
  panelId?: string;
}) => void;

export type TableRowActionHandler<Row> = (action: ActionViewModel, context: {
  tableId: string;
  row: Row;
  rowId?: string;
}) => void;

export type TableBulkActionHandler<Row> = (action: ActionViewModel, context: {
  tableId: string;
  scope: "visible" | "selected" | "allMatching";
  rows: Row[];
  selectedRowIds: string[];
}) => void;

export type FormActionHandler<Values> = (action: ActionViewModel, context: {
  formId: string;
  values: Values;
}) => void;

export type CalendarCellActionHandler = (action: ActionViewModel, context: {
  calendarId: string;
  date: string;
}) => void;
```

Common visual props

Most widgets should share these props:

```
export interface CommonWidgetProps {
  id?: string;
  className?: string;
  style?: React.CSSProperties;
  density?: "compact" | "normal" | "spacious";
  tone?: "neutral" | "success" | "warning" | "danger" | "info";
  dataAttributes?: Record<string, string | number | boolean>;
}
```

Renderer adapters may derive dataAttributes from node.meta using the current dataAttrs pattern from web/src/admin-dsl/renderUtils.ts.

Target File Layout

Use one directory per widget. Every widget directory should include at least a component file and a Storybook file. More complex widgets can include types, tests, parts, and fixtures.

```

web/src/admin-dsl/widgets/
  atoms/
    ActionButton/
      ActionButton.tsx
      ActionButton.stories.tsx
      ActionButton.types.ts
      index.ts
    StatusText/
      StatusText.tsx
      StatusText.stories.tsx
      StatusText.types.ts
      index.ts
  molecules/
    ActionGroup/
      ActionGroup.tsx
      ActionGroup.stories.tsx
      ActionGroup.types.ts
      index.ts
  organisms/
    ResourceTable/
      ResourceTable.tsx
      ResourceTable.stories.tsx
      ResourceTable.types.ts
      ResourceTable.test.tsx
      ResourceTable.fixtures.ts
      index.ts

```

Renderer adapters should live separately from the widgets so widgets remain reusable.

```

web/src/admin-dsl/rendering/
  AdminDslRenderer.tsx
  adapterTypes.ts
  registry.ts
  adapters/
    pageHeaderAdapter.ts
    panelAdapter.ts
    resourceTableAdapter.ts
    comparisonTableAdapter.ts
    monthCalendarAdapter.ts
    formAdapter.ts

```

The adapter layer consumes raw Admin DSL nodes and produces widget props. The widget layer should not know how AdminNode.props is encoded.

Widget Catalog Summary

This table maps current Admin DSL constructs to proposed widgets. Detailed notes follow in later sections.

Current construct	Proposed widget(s)	Class	Primary action slots	File directory
AdminPageRendered default shell	DefaultAdminShell Admin-PageChrome	Organism	page-level side surfaces	organisms/DefaultAdminShell

Current construct	Proposed widget(s)	Class	Primary action slots	File directory
shell.kind=admin, variant=workbench	WorkbenchShell, WorkbenchSidebar, WorkbenchTopbar	Organism	sidebarNav	organisms/WorkbenchShell/
pageHeader	PageHeader, Breadcrumbs, PageTitleBlock	Organism + molecules	primaryActions	organisms/PageHeader/
dashboardGrid	DashboardGrid, DashboardGridItem	Organism	none	organisms/DashboardGrid/
toolbar	Toolbar, ActionGroup	Molecule	toolbarActions	molecules/Toolbar/
panel	Panel, PanelHeader, PanelBody, PanelFooter	Organism	toolbar, footer	organisms/Panel/
splitPane	SplitPane	Organism	none	organisms/SplitPane/
tabs	Tabs, TabButton	Molecule	tabChange	molecules/Tabs/
filterBar	FilterBar, FilterPill	Molecule	filterChange	molecules/FilterBar/
searchBox	SearchBox	Molecule	searchSubmit	molecules/SearchBox/
previewFrame	PreviewFrame	Organism	previewActions	organisms/PreviewFrame/
metricCard	MetricCard	Molecule	none	molecules/MetricCard/
statusBadge	StatusText / StatusBadge	Atom	none	atoms/StatusText/
comparisonTable	ComparisonTable	Organism	rowActions	organisms/ComparisonTable/
monthCalendar	MonthCalendar, CalendarDayButton, CalendarLegend	Organism + atoms	previousMonth, nextMonth, selectDate	organisms/MonthCalendar/
resourceTable	ResourceTable, ResourceTableRow, ResourceTableCell, BulkActionBar, PaginationBar	Organism	rowActions, rowOverflow, bulkToolbar, pagination	organisms/ResourceTable/
emptyState	EmptyState	Molecule	primaryAction	molecules/EmptyState/
loadingState	LoadingState	Molecule	none	molecules/LoadingState/
inlineError	InlineError	Molecule	none	molecules/InlineError/
kvList	KeyValueList	Molecule	none	molecules/KeyValueList/
markdownBlock	MarkdownBlock	Molecule	none	molecules/MarkdownBlock/
activityFeed	ActivityFeed, ActivityFeedItem	Molecule	item action later	molecules/ActivityFeed/
imageGrid	ImageGrid, ImageCard	Organism + molecule	card action later	organisms/ImageGrid/
imageGallery	ImageGallery, GalleryImageCard	Organism + molecule	imageOpen	organisms/ImageGallery/

Current construct	Proposed widget(s)	Class	Primary action slots	File directory
modal, drawer, sheet, detailPanel, inlinePanel	OverlaySurface, SurfaceHeader, SurfaceBody	Organism	close, footer actions later	organisms/OverlaySurface/
confirmDialog form	ConfirmDialog AdminForm, FormLifecycleBanner, FormErrorSummary	Organism Organism + molecules	confirm, cancel submit, cancel, formFooter	organisms/ConfirmDialog/organisms/AdminForm/
fieldGroup field nodes	FieldGroup TextField, TextareaField, MoneyField, DurationField, DateField, TimeField, SelectField, SwitchField, ImageField	Molecule Molecules + atoms	none field change later	molecules/FieldGroup/ molecules/*Field/
saveBar calendarWeek	SaveBar CalendarWeek, CalendarGrid, CalendarAgenda	Molecule Organism	primary child block actions	molecules/SaveBar/organisms/CalendarWeek/
appointmentBlock availability-Block, timeOffBlock	CalendarEventBlock	Molecule	eventOpen	molecules/CalendarEventBlock/
resourcePage, resourceDetail, actionMenu	future widgets or remove from schema	TBD	TBD	TBD

Shell Widgets

WorkbenchShell

Current source: WorkbenchShell in web/src/admin-dsl/render.tsx.

Classification: Organism, role shell.

Purpose: Render the information-dense admin workbench frame: sidebar on desktop, sticky topbar on mobile, global background, content max width, and page body region.

Props:

```
export interface WorkbenchShellProps extends CommonWidgetProps {
  pageId: string;
  title: string;
  sidebar: SidebarNavProps;
  user?: WorkbenchUser;
  children: React.ReactNode;
}
```

```
export interface WorkbenchUser {
  name: string;
  role?: string;
  initials?: string;
}
```

Action slots:

```
onSidebarAction?: (action: ActionViewModel, context: {
  item: SidebarNavItem;
  activeItemId?: string;
}) => void;
```

Usage example:

```
<WorkbenchShell
  pageId="admin-intake"
  title="Fringe Admin"
  sidebar={{ activeItemId: "requests", items }}
  user={{ name: "Admin User", role: "Administrator", initials: "AD" }}
  onSidebarAction={handleSidebarAction}
>
  <PageHeader title="Request Triage" />
  <DashboardGrid>{ /* panels */}</DashboardGrid>
</WorkbenchShell>
```

Storybook stories:

- DefaultDesktop: sidebar visible, active item selected.
- MobileTopbar: mobile topbar visible, sidebar hidden.
- LongNavigation: many nav items, footer user visible.
- NoUser: no sidebar user block.
- ActionDispatch: clicking nav item logs contextual sidebar action.

File layout:

```
web/src/admin-dsl/widgets/organisms/WorkbenchShell/
  WorkbenchShell.tsx
  WorkbenchShell.types.ts
  WorkbenchShell.stories.tsx
  WorkbenchShell.test.tsx
  index.ts
```

DefaultAdminShell

Current source: fallback branch in AdminPageRenderer in web/src/admin-dsl/render.tsx.

Classification: Organism, role shell.

Purpose: Render non-workbench Admin DSL pages with optional side surfaces. This keeps the legacy/default page shell separate from workbench chrome.

Props:

```
export interface DefaultAdminShellProps extends CommonWidgetProps {
  pageId: string;
  shellKind: "admin" | "dashboard" | "resource" | "calendar" | "settings" | "bare";
  eyebrow?: string;
```

```

    title: string;
    description?: string;
    main: React.ReactNode;
    side?: React.ReactNode;
  }

```

Action slots: None directly. Child widgets own actions.

Storybook stories:

- MainOnly: page without side surfaces.
- WithSideSurfaces: drawers/modals rendered in side column.
- CalendarShell: calendar background variant.
- MobileSideColumn: side column stacks below main content.

File layout:

```

web/src/admin-dsl/widgets/organisms/DefaultAdminShell/
  DefaultAdminShell.tsx
  DefaultAdminShell.types.ts
  DefaultAdminShell.stories.tsx
  index.ts

```

Action Widgets

ActionButton

Current source: renderActions button branch in web/src/admin-dsl/render.tsx plus row/bulk/search special buttons.

Classification: Atom.

Purpose: Render one action with consistent disabled, loading, primary, danger, subtle, and placement-aware styling.

Props:

```

export interface ActionButtonProps<C = unknown> extends CommonWidgetProps {
  action: ActionViewModel;
  context?: C;
  variant?: "solid" | "soft" | "subtle" | "danger" | "overflow";
  size?: "sm" | "md" | "touch";
  onAction?: (action: ActionViewModel, context: C) => void;
}

```

Action slots: The widget executes the action it receives through onAction.

Storybook stories:

- Primary
- Secondary
- Danger
- SubtleLink
- Loading
- Disabled
- TouchTargetMobile

File layout:

```
web/src/admin-dsl/widgets/atoms/ActionButton/  
  ActionButton.tsx  
  ActionButton.types.ts  
  ActionButton.stories.tsx  
  index.ts
```

ActionGroup

Current source: renderActions wrapper.

Classification: Molecule.

Purpose: Render a list of actions with consistent spacing, wrapping, and slot-specific visual treatment.

Props:

```
export interface ActionGroupProps<C = unknown> extends CommonWidgetProps {  
  actions: ActionViewModel[];  
  context?: C;  
  slot: "pageHeader" | "toolbar" | "panelToolbar" | "panelFooter" | "row" | "rowOverflow"  
  align?: "start" | "end" | "between";  
  onAction?: (action: ActionViewModel, context: C) => void;  
}
```

Storybook stories:

- PageHeaderActions
- PanelFooterActions
- RowActions
- BulkToolbarActions
- FormFooterActions
- OverflowActions
- WrappingMobile

File layout:

```
web/src/admin-dsl/widgets/molecules/ActionGroup/  
  ActionGroup.tsx  
  ActionGroup.types.ts  
  ActionGroup.stories.tsx  
  index.ts
```

OverflowActionButton

Current source: renderTableCell branch for kind === "overflowActions".

Classification: Atom.

Purpose: Render the ellipsis affordance for row overflow menus. The first implementation can dispatch the first action, matching current behavior. A later implementation can open a real menu.

Props:

```
export interface OverflowActionButtonProps<C = unknown> {  
  label?: string;  
  actions: ActionViewModel[];  
  context: C;
```

```
    onAction?: (action: ActionViewModel, context: C) => void;
}
```

Storybook stories:

- SingleOverflowAction
- NoActionsDisabled
- FutureMenuPreview

File layout:

```
web/src/admin-dsl/widgets/atoms/OverflowActionButton/
  OverflowActionButton.tsx
  OverflowActionButton.types.ts
  OverflowActionButton.stories.tsx
  index.ts
```

Layout Widgets

PageHeader

Current source: case "pageHeader" in renderAdminNode.

Classification: Organism.

Props:

```
export interface PageHeaderProps extends CommonWidgetProps {
  breadcrumbs?: string[];
  title: string;
  description?: string;
  primaryActions?: ActionViewModel[];
  onPrimaryAction?: PageActionHandler;
}
```

Action slots: primaryActions with page context.

Usage example:

```
<PageHeader
  breadcrumbs={['Admin DSL', 'Workbench v2']}
  title="Request Triage"
  description="Review customer intake requests."
  primaryActions={[newServiceAction]}
  onPrimaryAction={handlePageAction}
/>
```

Storybook stories:

- Default
- WithBreadcrumbs
- WithPrimaryAction
- LongTitleMobile
- NoDescription

File layout:

```
web/src/admin-dsl/widgets/organisms/PageHeader/
  PageHeader.tsx
```

PageHeader.types.ts
PageHeader.stories.tsx
PageHeader.test.tsx
index.ts

DashboardGrid

Current source: case "dashboardGrid" and layoutSpan/layoutOrder helpers.

Classification: Organism.

Props:

```
export interface DashboardGridProps extends CommonWidgetProps {  
  columns?: { desktop?: number; tablet?: number; mobile?: number };  
  gap?: "compact" | "normal" | "spacious";  
  children: React.ReactNode;  
}
```

```
export interface DashboardGridItemProps {  
  span?: { desktop?: number; tablet?: number; mobile?: number };  
  order?: number;  
  children: React.ReactNode;  
}
```

Action slots: None.

Storybook stories:

- TwelveColumnDesktop
- CompactGap
- MixedSpanCards
- MobileSingleColumn
- Ordering

File layout:

web/src/admin-dsl/widgets/organisms/DashboardGrid/
DashboardGrid.tsx
DashboardGrid.types.ts
DashboardGrid.stories.tsx
index.ts

Panel

Current source: case "panel" in renderAdminNode.

Classification: Organism.

Props:

```
export interface PanelProps extends CommonWidgetProps {  
  title?: string;  
  subtitle?: string;  
  eyebrow?: string;  
  body?: string;  
  padding?: "none" | "normal";  
  toolbarActions?: ActionViewModel[];
```

```

    footerActions?: ActionViewModel[];
    children?: React.ReactNode;
    onToolbarAction?: PanelActionHandler;
    onFooterAction?: PanelActionHandler;
}

```

Action slots: toolbarActions, footerActions.

Usage example:

```

<Panel
  title="Today's queue"
  density="compact"
  padding="none"
  footerActions={[assignAction]}
  onFooterAction={handlePanelFooterAction}
>
  <ResourceTable {...tableProps} />
</Panel>

```

Storybook stories:

- Default
- CompactNoPadding
- WithToolbarAction
- WithFooterActions
- BodyOnly
- NestedResourceTable
- MobilePanelPadding

File layout:

```

web/src/admin-dsl/widgets/organisms/Panel/
  Panel.tsx
  Panel.types.ts
  Panel.stories.tsx
  Panel.test.tsx
  index.ts

```

Toolbar

Current source: case "toolbar".

Classification: Molecule.

Props:

```

export interface ToolbarProps extends CommonWidgetProps {
  actions: ActionViewModel[];
  onAction?: PageActionHandler;
}

```

Action slots: toolbarActions.

Storybook stories:

- Default
- ManyActionsWrap
- MobileTouchTargets

File layout:

```
web/src/admin-dsl/widgets/molecules/Toolbar/  
  Toolbar.tsx  
  Toolbar.types.ts  
  Toolbar.stories.tsx  
  index.ts
```

SplitPane

Current source: case "splitPane".

Classification: Organism.

Props:

```
export interface SplitPaneProps extends CommonWidgetProps {  
  leftWidth?: string;  
  rightWidth?: string;  
  gap?: number;  
  children: React.ReactNode;  
}
```

Action slots: None.

Storybook stories:

- MasterDetail
- TwoPanels
- MobileStacked

File layout:

```
web/src/admin-dsl/widgets/organisms/SplitPane/  
  SplitPane.tsx  
  SplitPane.types.ts  
  SplitPane.stories.tsx  
  index.ts
```

Tabs

Current source: case "tabs".

Classification: Molecule.

Props:

```
export interface TabsProps extends CommonWidgetProps {  
  tabs: Array<{ id: string; label: string }>;  
  value?: string;  
  action?: ActionViewModel;  
  onTabChange?: (action: ActionViewModel | undefined, context: { tab: { id: string; label:  
}
```

Action slots: tabChange.

Storybook stories:

- Default
- ActiveTab

- NoActionReadonly
- WrappingMobile

File layout:

```
web/src/admin-dsl/widgets/molecules/Tabs/
  Tabs.tsx
  Tabs.types.ts
  Tabs.stories.tsx
  index.ts
```

FilterBar

Current source: case "filterBar".

Classification: Molecule.

Props:

```
export interface FilterBarProps extends CommonWidgetProps {
  filters: Array<{ id: string; label: string }>;
  value?: string;
  action?: ActionViewModel;
  onFilterChange?: (action: ActionViewModel | undefined, context: { filter: { id: string;
}
```

Action slots: filterChange.

Storybook stories:

- Default
- ActiveFilter
- ManyFiltersWrap
- Readonly

File layout:

```
web/src/admin-dsl/widgets/molecules/FilterBar/
  FilterBar.tsx
  FilterBar.types.ts
  FilterBar.stories.tsx
  index.ts
```

SearchBox

Current source: case "searchBox".

Classification: Molecule.

Props:

```
export interface SearchBoxProps extends CommonWidgetProps {
  label?: string;
  placeholder?: string;
  value?: string;
  action?: ActionViewModel;
  onSearch?: (action: ActionViewModel | undefined, context: { query: string }) => void;
}
```

Action slots: searchSubmit.

Storybook stories:

- Default
- WithInitialValue
- NoAction
- SubmitDispatch

File layout:

```
web/src/admin-dsl/widgets/molecules/SearchBox/  
  SearchBox.tsx  
  SearchBox.types.ts  
  SearchBox.stories.tsx  
  index.ts
```

Resource Widgets

ResourceTable

Current source: case "resourceTable", renderTableCell, bulk bar, pagination, and mobile table CSS in responsiveCss.

Classification: Organism.

Purpose: Render semantic resource rows as a desktop table and mobile card list. This is the most important information-dense Admin DSL widget.

Props:

```
export type ResourceTableColumnKind =  
  | "text"  
  | "badge"  
  | "boolean"  
  | "dragHandle"  
  | "actions"  
  | "overflowActions"  
  | "money"  
  | "relativeTime";
```

```
export interface ResourceTableColumn<Row = Record<string, unknown>> {  
  id: string;  
  accessor?: keyof Row | string;  
  label?: string;  
  kind?: ResourceTableColumnKind;  
  primary?: boolean;  
  tone?: string;  
  width?: number | string;  
  map?: Record<string, { label: string; tone?: string }>;  
}
```

```
export interface ResourceTableProps<Row = Record<string, unknown>> extends CommonWidgetPro  
  tableId: string;  
  columns: ResourceTableColumn<Row>[];  
  rows: Row[];  
  selectable?: boolean;
```

```

selectedRowIds?: string[];
bulkLabel?: string;
empty?: React.ReactNode;
rowActions?: ActionViewModel[];
bulkActions?: ActionViewModel[];
pagination?: {
  page: number;
  total: number;
  actions?: ActionViewModel[];
};
onRowAction?: TableRowActionHandler<Row>;
onBulkAction?: TableBulkActionHandler<Row>;
onSelectionChange?: (context: { tableId: string; selectedRowIds: string[] }) => void;
}

```

Action slots:

- rowActions: receives { tableId, row, rowId }.
- rowOverflow: same context as row actions, displayed as overflow affordance.
- bulkToolbar: receives { tableId, scope, rows, selectedRowIds }.
- pagination: receives table/page context.

Usage example:

```

<ResourceTable
  tableId="requests"
  columns={requestColumns}
  rows={requestRows}
  selectable
  bulkLabel="3 visible requests"
  bulkActions={[assignAction]}
  onRowAction={(action, ctx) => dispatch(action, ctx)}
  onBulkAction={(action, ctx) => dispatch(action, ctx)}
/>

```

Storybook stories:

- DefaultRows
- SelectableWithBulkToolbar
- RowActions
- OverflowActions
- EmptyState
- Pagination
- StatusBadgeCells
- DragHandleColumn
- MobileCards
- LongTextAndDenseRows
- ActionDispatchContexts

File layout:

```

web/src/admin-dsl/widgets/organisms/ResourceTable/
  ResourceTable.tsx
  ResourceTable.types.ts
  ResourceTable.stories.tsx
  ResourceTable.test.tsx

```

```
ResourceTable.fixtures.ts
parts/
  BulkActionBar.tsx
  PaginationBar.tsx
  ResourceTableCell.tsx
  ResourceTableRow.tsx
index.ts
```

ResourceTableCell

Current source: renderTableCell.

Classification: Molecule.

Props:

```
export interface ResourceTableCellProps<Row = Record<string, unknown>> {
  column: ResourceTableColumn<Row>;
  row: Row;
  tableId: string;
  onRowAction?: TableRowActionHandler<Row>;
}
```

Storybook stories:

- TextCell
- PrimaryTextCell
- MutedTextCell
- BadgeMappedCell
- BooleanCell
- DragHandleCell
- ActionsCell
- OverflowCell

File layout: inside ResourceTable/parts/ unless promoted to standalone widget later.

BulkActionBar

Current source: bulk action bar inside resourceTable branch.

Classification: Molecule.

Props:

```
export interface BulkActionBarProps<Row = Record<string, unknown>> {
  tableId: string;
  label?: string;
  rows: Row[];
  selectedRowIds: string[];
  actions: ActionViewModel[];
  onBulkAction?: TableBulkActionHandler<Row>;
}
```

Storybook stories:

- VisibleRowsScope
- SelectedRowsScope
- PrimaryBulkAction

- DangerBulkAction
- NoSelectionDisabled

File layout: organisms/ResourceTable/parts/BulkActionBar.tsx.

PaginationBar

Current source: pagination footer inside resourceTable branch.

Classification: Molecule.

Props:

```
export interface PaginationBarProps {
  page: number;
  total: number;
  actions?: ActionViewModel[];
  onAction?: (action: ActionViewModel, context: { page: number; total: number }) => void;
}
```

Storybook stories:

- Default
- WithNextPrevious
- ManyResults

File layout: organisms/ResourceTable/parts/PaginationBar.tsx.

Data Display Widgets

MetricCard

Current source: case "metricCard".

Classification: Molecule.

Props:

```
export interface MetricCardProps extends CommonWidgetProps {
  label: string;
  value: string | number;
  caption?: string;
}
```

Storybook stories:

- Default
- SuccessTone
- WarningTone
- DangerTone
- LongCaption

File layout:

```
web/src/admin-dsl/widgets/molecules/MetricCard/
  MetricCard.tsx
  MetricCard.types.ts
  MetricCard.stories.tsx
  index.ts
```

StatusText / StatusBadge

Current source: renderTableCell badge branch and schema statusBadge. The current statusBadge node has no explicit render branch and falls back to JSON output.

Classification: Atom.

Props:

```
export interface StatusTextProps extends CommonWidgetProps {
  label: string;
  tone?: "neutral" | "success" | "warning" | "danger";
  variant?: "text" | "pill";
}
```

Storybook stories:

- NeutralText
- SuccessText
- WarningText
- DangerText
- PillVariant
- MappedFromTableValue

File layout:

```
web/src/admin-dsl/widgets/atoms/StatusText/
  StatusText.tsx
  StatusText.types.ts
  StatusText.stories.tsx
  index.ts
```

ComparisonTable

Current source: case "comparisonTable".

Classification: Organism.

Props:

```
export interface ComparisonTableRow {
  id?: string;
  field: string;
  current?: string;
  draft?: string;
  scheduled?: string;
  actions?: ActionViewModel[];
}
```

```
export interface ComparisonTableProps extends CommonWidgetProps {
  tableId: string;
  rows: ComparisonTableRow[];
  empty?: React.ReactNode;
  onRowAction?: (action: ActionViewModel, context: { tableId: string; row: ComparisonTable
}
```

Action slots: rowActions.

Storybook stories:

- DefaultChanges
- NoChangesEmptyState
- RowActions
- MobileCards
- LongValues

File layout:

```
web/src/admin-dsl/widgets/organisms/ComparisonTable/
  ComparisonTable.tsx
  ComparisonTable.types.ts
  ComparisonTable.stories.tsx
  ComparisonTable.test.tsx
  index.ts
```

KeyValueList

Current source: case "kvList".

Classification: Molecule.

Props:

```
export interface KeyValueListProps extends CommonWidgetProps {
  items: Array<{ label: string; value: string | number | React.ReactNode }>;
  labelWidth?: number | string;
}
```

Storybook stories:

- Default
- LongLabels
- EmptyItems
- AuditDetails

File layout:

```
web/src/admin-dsl/widgets/molecules/KeyValueList/
  KeyValueList.tsx
  KeyValueList.types.ts
  KeyValueList.stories.tsx
  index.ts
```

ActivityFeed

Current source: case "activityFeed".

Classification: Molecule.

Props:

```
export interface ActivityFeedItem {
  time: string;
  title: string;
  body?: string;
  action?: ActionViewModel;
}
```

```
export interface ActivityFeedProps extends CommonWidgetProps {
  items: ActivityFeedItem[];
  onItemAction?: (action: ActionViewModel, context: { item: ActivityFeedItem }) => void;
}
```

Storybook stories:

- Default
- Empty
- LongBody
- AuditLog
- WithItemActions

File layout:

```
web/src/admin-dsl/widgets/molecules/ActivityFeed/
  ActivityFeed.tsx
  ActivityFeed.types.ts
  ActivityFeed.stories.tsx
  index.ts
```

MarkdownBlock

Current source: case "markdownBlock".

Classification: Molecule.

Props:

```
export interface MarkdownBlockProps extends CommonWidgetProps {
  markdown: string;
  tone?: "neutral" | "muted";
}
```

Storybook stories:

- PlainText
- Multiline
- Muted

File layout:

```
web/src/admin-dsl/widgets/molecules/MarkdownBlock/
  MarkdownBlock.tsx
  MarkdownBlock.types.ts
  MarkdownBlock.stories.tsx
  index.ts
```

EmptyState, LoadingState, InlineError

Current source: case "emptyState", case "loadingState", case "inlineError".

Classification: Molecules.

Props:

```
export interface EmptyStateProps extends CommonWidgetProps {
  title: string;
  body?: string;
```

```

    action?: ActionViewModel;
    onAction?: PageActionHandler;
}

export interface LoadingStateProps extends CommonWidgetProps {
  title?: string;
  body?: string;
}

export interface InlineErrorProps extends CommonWidgetProps {
  title: string;
  body?: string;
}

```

Storybook stories:

- EmptyState/Default
- EmptyState/WithAction
- LoadingState/Default
- LoadingState/WithBody
- InlineError/Default
- InlineError/LongMessage

File layout:

```

web/src/admin-dsl/widgets/molecules/EmptyState/
  EmptyState.tsx
  EmptyState.types.ts
  EmptyState.stories.tsx
  index.ts
web/src/admin-dsl/widgets/molecules/LoadingState/
  LoadingState.tsx
  LoadingState.types.ts
  LoadingState.stories.tsx
  index.ts
web/src/admin-dsl/widgets/molecules/InlineError/
  InlineError.tsx
  InlineError.types.ts
  InlineError.stories.tsx
  index.ts

```

Media Widgets

PreviewFrame

Current source: case "previewFrame".

Classification: Organism.

Props:

```

export interface PreviewFrameProps extends CommonWidgetProps {
  previewId: string;
  kicker?: string;
  title: string;
  body?: string;
  url?: string;
}

```

```

    height?: number;
    placeholder?: string;
    actions?: ActionViewModel[];
    onAction?: PanelActionHandler;
}

```

Action slots: previewActions.

Storybook stories:

- IframeConnected
- Placeholder
- WithBodyAndActions
- TallPreview
- MobilePreview

File layout:

```

web/src/admin-dsl/widgets/organisms/PreviewFrame/
  PreviewFrame.tsx
  PreviewFrame.types.ts
  PreviewFrame.stories.tsx
  index.ts

```

ImageGrid

Current source: case "imageGrid".

Classification: Organism.

Props:

```

export interface ImageGridItem {
  id?: string;
  title: string;
  subtitle?: string;
  status?: string;
  tone?: string;
  url?: string;
}

```

```

export interface ImageGridProps extends CommonWidgetProps {
  items: ImageGridItem[];
}

```

Action slots: None in current renderer. A future cardAction slot can be added.

Storybook stories:

- Default
- WithStatuses
- Empty
- ResponsiveGrid

File layout:

```

web/src/admin-dsl/widgets/organisms/ImageGrid/
  ImageGrid.tsx
  ImageGrid.types.ts

```

```
ImageGrid.stories.tsx
parts/ImageCard.tsx
index.ts
```

ImageGallery

Current source: case "imageGallery".

Classification: Organism.

Props:

```
export interface GalleryImage {
  id?: string;
  slot?: string;
  title?: string;
  subtitle?: string;
  status?: string;
  tone?: string;
  url?: string;
  alt?: string;
}

export interface ImageGalleryProps extends CommonWidgetProps {
  galleryId: string;
  images: GalleryImage[];
  emptyText?: string;
  imageAction?: ActionViewModel;
  onImageAction?: (action: ActionViewModel, context: { galleryId: string; image: GalleryImage }) => void;
}
```

Action slots: imageOpen.

Storybook stories:

- DefaultPhotos
- MissingPhoto
- EmptyGallery
- ClickableImages
- MobileGrid

File layout:

```
web/src/admin-dsl/widgets/organisms/ImageGallery/
  ImageGallery.tsx
  ImageGallery.types.ts
  ImageGallery.stories.tsx
  parts/GalleryImageCard.tsx
  index.ts
```

Calendar Widgets

MonthCalendar

Current source: case "monthCalendar" and buildMonthCells.

Classification: Organism.

Props:

```
export interface MonthCalendarMarker {
  date: string;
  kind: string;
  tone?: "success" | "warning" | "danger" | "neutral";
}

export interface MonthCalendarLegendItem {
  kind: string;
  label: string;
  tone?: "success" | "warning" | "danger" | "neutral";
}

export interface MonthCalendarProps extends CommonWidgetProps {
  calendarId: string;
  month: string; // YYYY-MM
  label?: string;
  selectedDate?: string;
  markers?: MonthCalendarMarker[];
  legend?: MonthCalendarLegendItem[];
  previousMonthAction?: ActionViewModel;
  nextMonthAction?: ActionViewModel;
  selectDateAction?: ActionViewModel;
  onMonthAction?: CalendarCellActionHandler;
  onSelectDate?: CalendarCellActionHandler;
}
```

Action slots: previousMonth, nextMonth, selectDate.

Storybook stories:

- DefaultMonth
- WithPublishedMarkers
- WithScheduledMarkers
- SelectedDate
- NoActionsReadonly
- PreviousNextActions
- MobileCalendar

File layout:

```
web/src/admin-dsl/widgets/organisms/MonthCalendar/
  MonthCalendar.tsx
  MonthCalendar.types.ts
  MonthCalendar.stories.tsx
  MonthCalendar.test.tsx
  parts/CalendarDayButton.tsx
  parts/CalendarLegend.tsx
  index.ts
```

CalendarWeek

Current source: web/src/admin-dsl/calendar.tsx and case "calendarWeek".

Classification: Organism.

Props:

```
export interface CalendarWeekProps extends CommonWidgetProps {
  calendarId: string;
  days: string[];
  hours: string[];
  blocks: CalendarEventBlockProps[];
  onBlockAction?: (action: ActionViewModel, context: { block: CalendarEventBlockProps }) => void;
}
```

Action slots: child block eventOpen actions.

Storybook stories:

- DefaultWeek
- AppointmentsAndTimeOff
- NoAppointments
- MobileAgenda
- LongDayLabels

File layout:

```
web/src/admin-dsl/widgets/organisms/CalendarWeek/
  CalendarWeek.tsx
  CalendarWeek.types.ts
  CalendarWeek.stories.tsx
  parts/CalendarGrid.tsx
  parts/CalendarAgenda.tsx
  index.ts
```

CalendarEventBlock

Current source: appointmentBlock, availabilityBlock, timeOffBlock branches in render.tsx and calendar.tsx.

Classification: Molecule.

Props:

```
export interface CalendarEventBlockProps extends CommonWidgetProps {
  id: string;
  kind: "appointment" | "availability" | "timeOff";
  clientName?: string;
  title?: string;
  service?: string;
  status?: string;
  startsAt?: string;
  endsAt?: string;
  column?: number;
  row?: number;
  span?: number;
  action?: ActionViewModel;
  onAction?: (action: ActionViewModel, context: { blockId: string }) => void;
}
```

Action slots: eventOpen.

Storybook stories:

- Appointment
- Availability
- TimeOff
- CompactBlock
- AgendaItem

File layout:

```
web/src/admin-dsl/widgets/molecules/CalendarEventBlock/
  CalendarEventBlock.tsx
  CalendarEventBlock.types.ts
  CalendarEventBlock.stories.tsx
  index.ts
```

Form Widgets

AdminForm

Current source: case "form".

Classification: Organism.

Props:

```
export interface AdminFormProps<Values = Record<string, unknown>> extends CommonWidgetProp
  formId: string;
  title?: string;
  dirty?: boolean;
  pending?: boolean;
  state?: "idle" | "dirty" | "pending" | "success" | "error";
  errors?: Record<string, string>;
  actions?: ActionViewModel[];
  children: React.ReactNode;
  onFormAction?: FormActionHandler<Values>;
}
```

Action slots: submit, cancel, formFooter, currently represented through generic actions in the render branch.

Usage example:

```
<AdminForm
  formId="serviceForm"
  title="Highlights"
  dirty
  actions={[saveAction, cancelAction]}
  onFormAction={(action, context) => submit(action, context.values)}
>
  <FieldGroup title="Identity">
    <TextField name="label" label="Label" value="Highlights" />
  </FieldGroup>
</AdminForm>
```

Storybook stories:

- Default
- DirtyState
- PendingState

- SuccessState
- WithErrors
- SubmitActions
- NestedFieldGroups

File layout:

```
web/src/admin-dsl/widgets/organisms/AdminForm/
  AdminForm.tsx
  AdminForm.types.ts
  AdminForm.stories.tsx
  AdminForm.test.tsx
  parts/FormLifecycleBanner.tsx
  parts/FormErrorSummary.tsx
  index.ts
```

FieldGroup

Current source: case "fieldGroup".

Classification: Molecule.

Props:

```
export interface FieldGroupProps extends CommonWidgetProps {
  title: string;
  children: React.ReactNode;
}
```

Action slots: None.

Storybook stories:

- Default
- MultipleFields
- LongTitle

File layout:

```
web/src/admin-dsl/widgets/molecules/FieldGroup/
  FieldGroup.tsx
  FieldGroup.types.ts
  FieldGroup.stories.tsx
  index.ts
```

Field widgets

Current source: FieldPreview in render.tsx.

Classification: Molecules composed from input atoms.

Shared props:

```
export interface FieldBaseProps<T = unknown> extends CommonWidgetProps {
  name: string;
  label: string;
  value?: T;
  required?: boolean;
  disabled?: boolean;
```

```

    error?: string;
    helpText?: string;
    onChange?: (context: { name: string; value: T }) => void;
  }

```

Individual widgets:

Current construct	Widget	Additional props	Stories	File directory
textField	TextField	placeholder?	default, empty, disabled, error	molecules/TextField/
textareaField	TextareaField	rows?, placeholder?	default, long text, error	molecules/TextareaField/
moneyField	MoneyField	currency?, min?, max?	cents value, empty, error	molecules/MoneyField/
durationField	DurationField	unit?	minutes, disabled, error	molecules/DurationField/
dateField	DateField	min?, max?	date value, empty, error	molecules/DateField/
timeField	TimeField	step?	time value, empty, error	molecules/TimeField/
selectField	SelectField	options	default, many options, disabled	molecules/SelectField/
switchField	SwitchField	none	on, off, disabled	molecules/SwitchField/
imageField	ImageField	accept?, maxSize?	empty, uploading, with image	molecules/ImageField/

Each directory should contain:

```

WidgetName.tsx
WidgetName.types.ts
WidgetName.stories.tsx
index.ts

```

SaveBar

Current source: case "saveBar".

Classification: Molecule.

Props:

```

export interface SaveBarProps extends CommonWidgetProps {
  status: string;
  primaryAction?: ActionViewModel;
  onPrimaryAction?: FormActionHandler<Record<string, unknown>>;
}

```

Action slots: primary.

Storybook stories:

- Ready
- UnsavedChanges

- Saving
- PrimaryAction
- MobileStacked

File layout:

```
web/src/admin-dsl/widgets/molecules/SaveBar/
  SaveBar.tsx
  SaveBar.types.ts
  SaveBar.stories.tsx
  index.ts
```

Surface Widgets

OverlaySurface

Current source: shared branch for modal, drawer, sheet, detailPanel, inlinePanel.

Classification: Organism.

Props:

export type OverlaySurfaceKind = "modal" | "drawer" | "sheet" | "detailPanel" | "inlinePan

```
export interface OverlaySurfaceProps extends CommonWidgetProps {
  surfaceId: string;
  kind: OverlaySurfaceKind;
  title: string;
  open?: boolean;
  children: React.ReactNode;
  closeAction?: ActionViewModel;
  footerActions?: ActionViewModel[];
  onCloseAction?: (action: ActionViewModel, context: { surfaceId: string; kind: OverlaySur
  onFooterAction?: (action: ActionViewModel, context: { surfaceId: string; kind: OverlaySu
}
```

Action slots: close, footer.

Storybook stories:

- Modal
- Drawer
- Sheet
- DetailPanel
- InlinePanel
- ClosedDashedPreview
- WithFooterActions
- MobileSurface

File layout:

```
web/src/admin-dsl/widgets/organisms/OverlaySurface/
  OverlaySurface.tsx
  OverlaySurface.types.ts
  OverlaySurface.stories.tsx
  index.ts
```

ConfirmDialog

Current source: case "confirmDialog".

Classification: Organism.

Props:

```
export interface ConfirmDialogProps extends CommonWidgetProps {
  dialogId: string;
  title: string;
  body?: string;
  tone?: "neutral" | "danger";
  confirmLabel?: string;
  confirmAction?: ActionViewModel;
  cancelAction?: ActionViewModel;
  onConfirm?: (action: ActionViewModel, context: { dialogId: string }) => void;
  onCancel?: (action: ActionViewModel, context: { dialogId: string }) => void;
}
```

Action slots: confirm, cancel.

Storybook stories:

- Default
- Danger
- LongBody
- WithCancel
- DispatchConfirm

File layout:

```
web/src/admin-dsl/widgets/organisms/ConfirmDialog/
  ConfirmDialog.tsx
  ConfirmDialog.types.ts
  ConfirmDialog.stories.tsx
  ConfirmDialog.test.tsx
  index.ts
```

Unsupported or Ambiguous Current Constructs

Some constructs exist in schema.ts or builder.ts but are not first-class rendered widgets today.

Construct	Current status	Recommendation
statusBadge	In schema/builder, no explicit render branch in render.tsx.	Map to StatusText/StatusBadge atom or remove if table badge cells cover use cases.
resourcePage	In schema, not rendered as node. resource.page creates an AdminPageBuilder, not a node.	Treat as shell/page builder concept, not widget.

Construct	Current status	Recommendation
resourceDetail	Builder exists, no explicit render branch.	Map to OverlaySurface kind=detailPanel or create ResourceDetail organism later.
actionMenu	In schema, no explicit render branch.	Defer until OverflowActionButton grows into a real menu.
availabilityBlock	Rendered the same as appointment except not time-off warning.	Keep as CalendarEventBlock kind="availability".

The first implementation pass should either remove unsupported constructs from the new vocabulary or create explicit widget targets for them. Do not keep silent fallback behavior for production widgets.

Renderer Adapter Plan

The new renderer should be a small adapter registry instead of a giant switch that directly renders JSX.

```
export interface AdminNodeAdapter<N extends AdminNode = AdminNode, P = unknown> {
  kind: string;
  toProps(node: N, context: AdminRenderContext): P;
  render(props: P): React.ReactNode;
}
```

Example adapter:

```
export const resourceTableAdapter: AdminNodeAdapter<AdminNode, ResourceTableProps> = {
  kind: "resourceTable",
  toProps(node, context) {
    const props = node.props ?? {};
    return {
      tableId: String(props.id ?? node.meta?.id ?? "table"),
      columns: readColumns(props),
      rows: readRows(props),
      selectable: Boolean(props.selectable),
      bulkActions: readActions(props.bulkActions),
      rowActions: readActions(props.actions).filter((action) => action.placement === "row"),
      onRowAction: (action, rowContext) => dispatchAdminAction(context, node, action, rowContext),
      onBulkAction: (action, bulkContext) => dispatchAdminAction(context, node, action, bulkContext),
    };
  },
  render(props) {
    return <ResourceTable {...props} />;
  },
};
```

The adapter owns compatibility with current Admin DSL JSON. The widget owns visual rendering.

Storybook Scenario Matrix

The current `AdminDslWorkbench.stories.tsx` has scenario-level stories. Keep those as integration stories, but add widget-level stories in each widget directory.

Integration stories to keep

Current story	Purpose after refactor
TargetDesktop	Full workbench dashboard with all major widgets.
TargetMobile	Full dashboard mobile behavior.
ServiceOperations	Resource table with row overflow and metrics.
RequestTriage	Dense request table with bulk actions and queue health panel.
DraftReviewQueue	Comparison table and publish summary.
CalendarPublishing	Month calendar plus comparison table.
TypedFormWorkbench	Form fields inside panels.
EmptyLoadingErrorStates	State widgets inside panels.
AuditWorkbench	Activity feed plus key-value detail panel.
DenseMobileOperations	Resource table mobile card behavior.

Widget-level story requirements

Every widget should have at least:

- Default: normal production-like data.
- Empty or NoData when applicable.
- WithActions when the widget exposes an action slot.
- Loading or Pending when the widget has lifecycle state.
- Error when the widget can display errors.
- Mobile when layout changes at mobile breakpoints.
- LongContent when text wrapping or overflow is important.
- DispatchContext when the widget has contextual action callbacks.

Example ResourceTable stories

```
export const DefaultRows: Story = {
  args: { columns: requestColumns, rows: requestRows },
};
```

```
export const SelectableWithBulkToolbar: Story = {
  args: {
    columns: requestColumns,
    rows: requestRows,
    selectable: true,
    bulkActions: [assignAction],
  },
};
```

```
export const DispatchContext: Story = {
  args: {
```

```

    columns: requestColumns,
    rows: requestRows,
    rowActions: [reviewAction],
    onRowAction: (action, context) => console.log(action, context),
  },
};

```

Design Decisions

Decision 1: Widgets receive typed props, adapters read Admin DSL JSON

Widgets should not call `str`, `jsonArray`, or `jsonObject` on raw `AdminNode.props`. That work belongs in adapters. This creates clean React components and keeps backward compatibility logic outside visual code.

Decision 2: Action slots become callback props with contextual signatures

The current renderer passes generic `AdminActionRef` and raw values to `dispatchAdminAction`. New widgets should expose slot-specific callbacks such as `onRowAction`, `onBulkAction`, `onFormAction`, and `onSelectDate`. The adapter can lower those callbacks to the existing runtime dispatch.

Decision 3: Storybook is required per widget

A widget is not complete unless it has stories. Scenario-level workbench stories are not enough because they do not isolate edge cases.

Decision 4: Keep visual implementation hand-authored first

The meta-spec compiler can generate prop contracts, registries, stories, and file layouts. It should not generate full visual React code until the widget boundaries stabilize.

Decision 5: Unsupported schema constructs must be explicit

Constructs such as `statusBadge`, `resourceDetail`, and `actionMenu` should not silently fall through to JSON rendering. They should either become widgets or be removed from the supported vocabulary.

Alternatives Considered

Alternative 1: Continue improving `render.tsx`

This is the lowest-effort path. It keeps all behavior in one place. It does not solve the structural problem: no widget contracts, no isolated stories, and no clear generated target surface.

Alternative 2: Generate all React widgets directly from the DSL meta-spec

This is too ambitious as a first step. Information-dense UI needs visual tuning. The better first target is generated contracts and scaffolding, with hand-authored widgets.

Alternative 3: Build only generic primitives and no Admin-specific organisms

Generic primitives are useful, but Admin DSL needs semantic organisms such as ResourceTable, ComparisonTable, MonthCalendar, and Panel. Without those, the DSL falls back into generic div composition.

Alternative 4: Keep action placement strings as the only action model

Placement strings are useful for runtime transport, but they are weak at the widget boundary. Callback signatures should encode context. A table row action should receive row context because that is what makes it a row action.

Implementation Plan

Phase 1: Create widget scaffolds without changing runtime

Generate or create directories for atoms, molecules, and organisms. Start with ActionButton, ActionGroup, Panel, PageHeader, and ResourceTable.

Phase 2: Move visual code out of render.tsx one widget at a time

Replace render branches with adapters that call widgets. Keep the Admin DSL JSON shape unchanged.

Recommended first order:

1. ActionButton and ActionGroup
2. Panel
3. PageHeader
4. MetricCard, EmptyState, LoadingState, InlineError
5. ResourceTable
6. ComparisonTable
7. MonthCalendar
8. AdminForm and field widgets
9. WorkbenchShell

Phase 3: Add Storybook stories per widget

Each moved widget must include widget-level stories before the render branch is considered replaced.

Phase 4: Introduce adapter registry

Create web/src/admin-dsl/rendering/registry.ts and move raw Admin DSL prop extraction into adapter files.

Phase 5: Convert integration stories to use the new renderer

The existing AdminDslWorkbench.stories.tsx scenarios should still pass. Screenshots should be re-captured after major widget migrations.

Phase 6: Feed the catalog into meta-spec/codegen experiments

Once the widget catalog stabilizes, later passes can generate:

- widget file scaffolds;
- Storybook skeletons;
- renderer adapter stubs;
- action slot type definitions;
- docs and prop tables;
- Playwright scenario sketches.

Open Questions

- Should all field widgets live as separate molecules or share one `FieldControl` organism with typed variants?
- Should `ResourceTableCell` be a public molecule or an internal `ResourceTable` part?
- Should `StatusText` use text-only status by default, or should pill badges remain available as a variant?
- Should `OverlaySurface` cover modal, drawer, sheet, `detailPanel`, and `inlinePanel`, or should drawer/modal become separate organisms?
- Should `CalendarWeek` stay part of Admin DSL v2, or is `MonthCalendar` enough for the current admin rebuild?
- How strict should the adapter layer be when old nodes contain unexpected props?
- Should file scaffolding be generated from this Markdown catalog or from a structured YAML/JSON version of the same IR?

References

Current implementation files:

- `web/src/admin-dsl/render.tsx`
- `web/src/admin-dsl/calendar.tsx`
- `web/src/admin-dsl/schema.ts`
- `web/src/admin-dsl/builder.ts`
- `web/src/admin-dsl/actions.ts`
- `web/src/admin-dsl/renderUtils.ts`
- `web/src/admin-dsl/AdminDslWorkbench.stories.tsx`

Related design documents:

- `design-doc/07-ui-dsl-meta-spec-compiler-implementation-guide.md`
- `meta-dsl-comments.md`